ELECTRICTY BILLING SYSTEM

- TANVI NITIN SURVE

BATCH- T361 / RP

ELECTRICITY BILLING SYSTEM

An **Electricity Billing System** is a project that simplifies and automates the process of managing electricity consumption and billing operations. This system ensures the efficient handling of activities such as maintaining consumer records, recording meter readings, generating electricity bills, and tracking payment status.

This project demonstrates the implementation of an Electricity Billing System using **MySQL**. It includes designing and managing multiple database tables, performing data manipulation operations, and executing advanced SQL queries. The objective of this project is to showcase practical skills in **database design, data management, and querying** while developing a reliable and organized billing system.

PROJECT AIM

- **Consumer Management:**
  To maintain consumer records including personal details, contact information, and connection status.
- **Connection and Meter Management:**
  To manage electricity connections and meter details and link them accurately with consumers.
- **Meter Reading Management:**
  To record and store periodic meter readings and calculate the electricity units consumed.
- **Billing Management:**
  To generate electricity bills based on meter readings and applicable tariff rates.
- **Payment Management:**
  To track bill payments, payment modes, and payment status for each consumer.

OBJECTIVES

1. **Set up the Electricity Billing System Database:**
   Create and populate the database with tables for consumers, connections, meters, meter readings, bills, and payments.

2. **CRUD Operations:**
   Perform Create, Read, Update, and Delete operations on the data to maintain accurate records of consumers, meter readings, and billing information.

3. **Advanced SQL Queries:**
   Develop complex queries to analyse electricity consumption, generate bills, track payments, and retrieve specific information for reporting purposes.

ER Diagram For ELECTRICTY BILLING SYSTEM

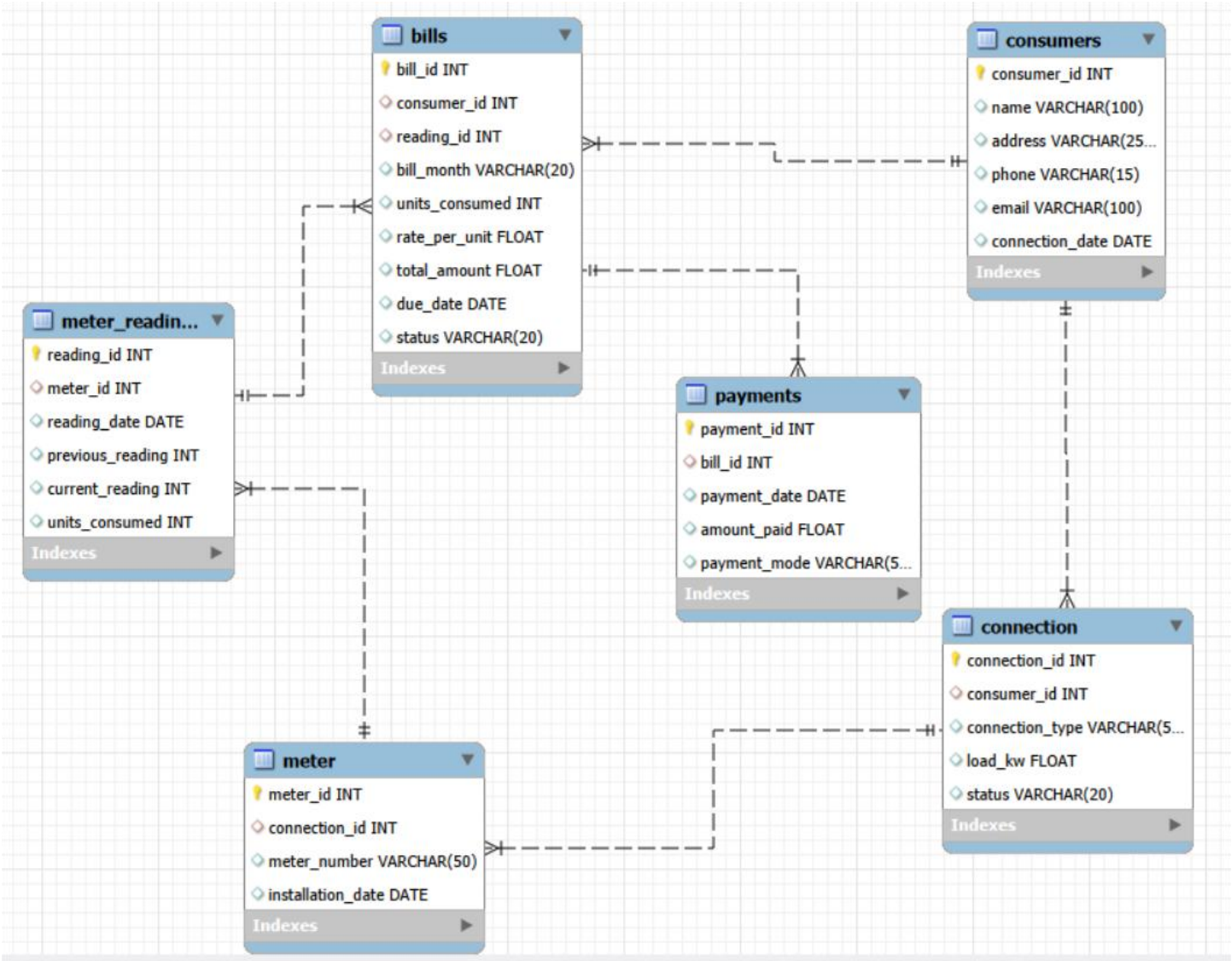
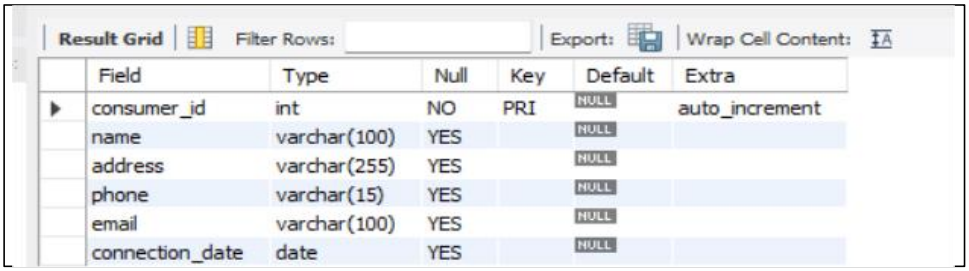


Table Description:

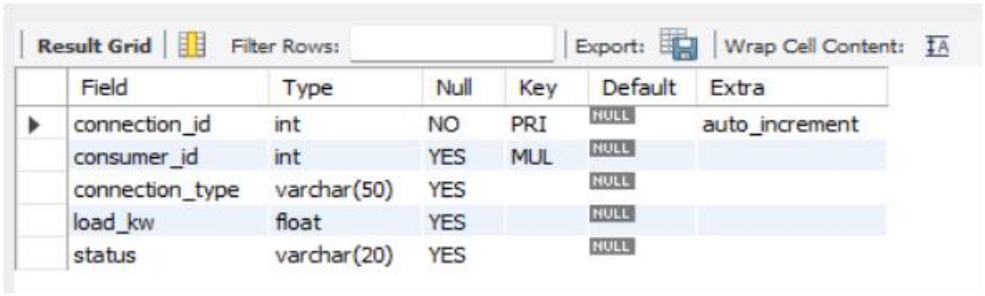
1) **Consumers**



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
consumer_id	int	NO	PRI	NULL	auto_increment
name	varchar(100)	YES		NULL	
address	varchar(255)	YES		NULL	
phone	varchar(15)	YES		NULL	
email	varchar(100)	YES		NULL	
connection_date	date	YES		NULL	

2) **Connection**



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

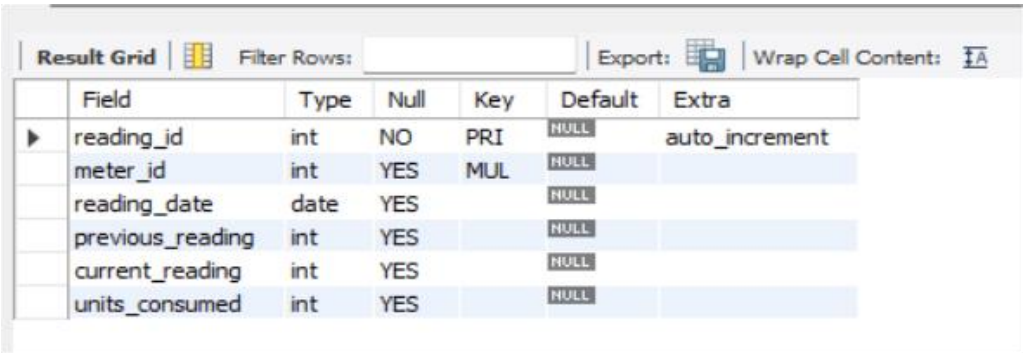
Field	Type	Null	Key	Default	Extra
connection_id	int	NO	PRI	NULL	auto_increment
consumer_id	int	YES	MUL	NULL	
connection_type	varchar(50)	YES		NULL	
load_kw	float	YES		NULL	
status	varchar(20)	YES		NULL	

3) **Meter**

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
meter_id	int	NO	PRI	NULL	auto_increment
connection_id	int	YES	MUL	NULL	
meter_number	varchar(50)	YES		NULL	
installation_date	date	YES		NULL	

4) **Meter Reading**



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
reading_id	int	NO	PRI	NULL	auto_increment
meter_id	int	YES	MUL	NULL	
reading_date	date	YES		NULL	
previous_reading	int	YES		NULL	
current_reading	int	YES		NULL	
units_consumed	int	YES		NULL	

5) Bill

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
bill_id	int	NO	PRI	NULL	auto_increment
consumer_id	int	YES	MUL	NULL	
reading_id	int	YES	MUL	NULL	
bill_month	varchar(20)	YES		NULL	
units_consumed	int	YES		NULL	
rate_per_unit	float	YES		NULL	
total_amount	float	YES		NULL	
due_date	date	YES		NULL	
status	varchar(20)	YES		NULL	

6) Payment

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
payment_id	int	NO	PRI	NULL	auto_increment
bill_id	int	YES	MUL	NULL	
payment_date	date	YES		NULL	
amount_paid	float	YES		NULL	
payment_mode	varchar(50)	YES		NULL	

CREATING DATABASE:

CREATE DATABASE electricity_billing_system;
use electricity_billing_system;

Table Creation & Insertion Commands:

1) Create Table Consumer

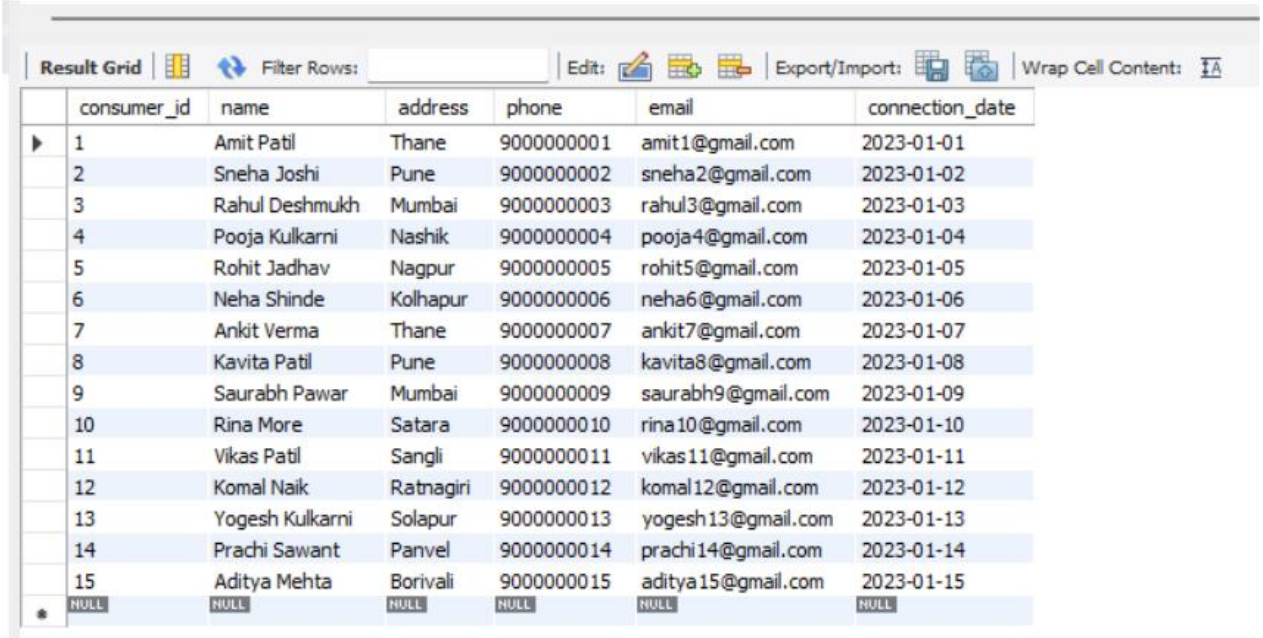
CREATE TABLE
Consumer(consumer_id INT PRIMARY
KEY AUTO_INCREMENT, name VARCHAR(100),
address VARCHAR(255),
phone VARCHAR(15),
email VARCHAR(100),
connection_date DATE);

Inserting Values into Consumers:

INSERT INTO Consumers (name, address, phone, email, connection_date)
VALUES
('Amit Patil','Thane','9000000001','amit1@gmail.com','2023-01-01'),
('Sneha Joshi','Pune','9000000002','sneha2@gmail.com','2023-01-02'),
('Rahul Deshmukh','Mumbai','9000000003','rahul3@gmail.com','2023-01-03'),

SELECT * FROM Consumers;

OUTPUT:


Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

consumer_id	name	address	phone	email	connection_date
1	Amit Patil	Thane	9000000001	amit1@gmail.com	2023-01-01
2	Sneha Joshi	Pune	9000000002	sneha2@gmail.com	2023-01-02
3	Rahul Deshmukh	Mumbai	9000000003	rahul3@gmail.com	2023-01-03
4	Pooja Kulkarni	Nashik	9000000004	pooja4@gmail.com	2023-01-04
5	Rohit Jadhav	Nagpur	9000000005	rohit5@gmail.com	2023-01-05
6	Neha Shinde	Kolhapur	9000000006	neha6@gmail.com	2023-01-06
7	Ankit Verma	Thane	9000000007	ankit7@gmail.com	2023-01-07
8	Kavita Patil	Pune	9000000008	kavita8@gmail.com	2023-01-08
9	Saurabh Pawar	Mumbai	9000000009	saurabh9@gmail.com	2023-01-09
10	Rina More	Satara	9000000010	rina10@gmail.com	2023-01-10
11	Vikas Patil	Sangli	9000000011	vikas11@gmail.com	2023-01-11
12	Komal Naik	Ratnagiri	9000000012	komal12@gmail.com	2023-01-12
13	Yogesh Kulkarni	Solapur	9000000013	yogesh13@gmail.com	2023-01-13
14	Prachi Sawant	Panvel	9000000014	prachi14@gmail.com	2023-01-14
15	Aditya Mehta	Borivali	9000000015	aditya15@gmail.com	2023-01-15
NULL	NULL	NULL	NULL	NULL	NULL

2) Create Table Connection

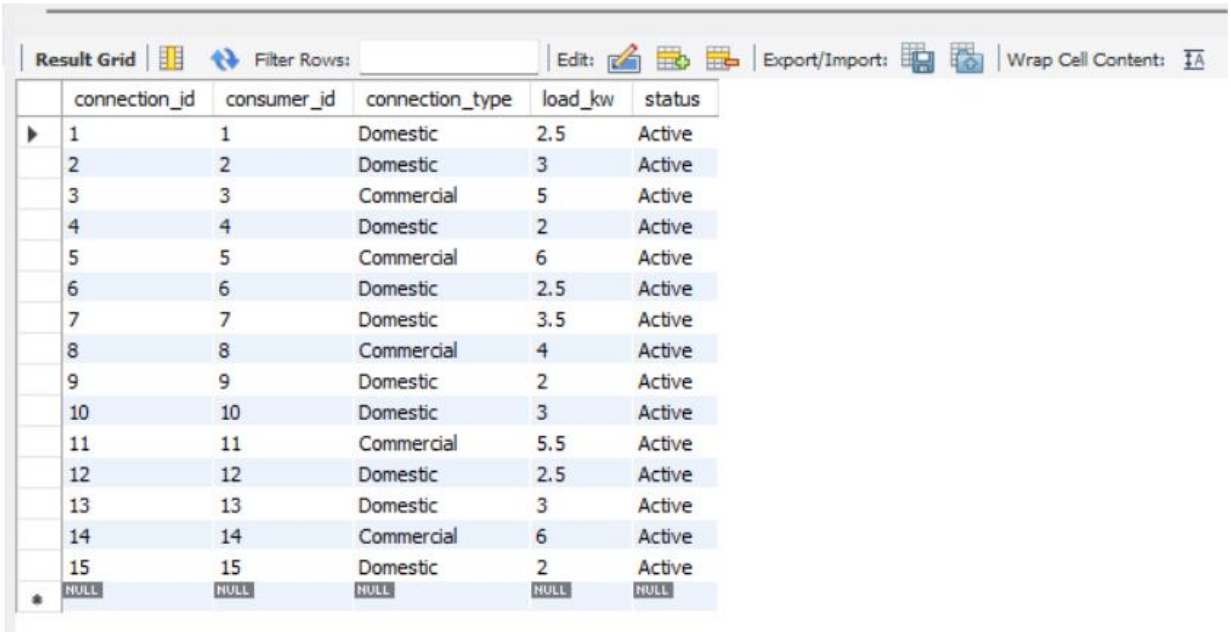
```
CREATE TABLE Connection
(                                                    connection_id INT PRIMARY KEY
AUTO_INCREMENT,                                      consumer_id INT,
connection_type VARCHAR(50),
load_kw FLOAT,
status VARCHAR(20),
FOREIGN KEY (consumer_id) REFERENCES Consumers(consumer_id)
);
```

Inserting Values into Connection:

```
INSERT INTO Connection (consumer_id, connection_type, load_kw, status)
VALUES
(1,'Domestic',2.5,'Active'),
(2,'Domestic',3.0,'Active'),
(3,'Commercial',5.0,'Active'),
(4,'Domestic',2.0,'Active'),
.
.
```

SELECT * FROM Connection;

OUTPUT:



connection_id	consumer_id	connection_type	load_kw	status
1	1	Domestic	2.5	Active
2	2	Domestic	3	Active
3	3	Commercial	5	Active
4	4	Domestic	2	Active
5	5	Commercial	6	Active
6	6	Domestic	2.5	Active
7	7	Domestic	3.5	Active
8	8	Commercial	4	Active
9	9	Domestic	2	Active
10	10	Domestic	3	Active
11	11	Commercial	5.5	Active
12	12	Domestic	2.5	Active
13	13	Domestic	3	Active
14	14	Commercial	6	Active
15	15	Domestic	2	Active
NULL	NULL	NULL	NULL	NULL

3) Create Table Meter

```
CREATE TABLE Meter
(                                                    meter_id INT PRIMARY KEY
AUTO_INCREMENT,                                      connection_id INT,
meter_number VARCHAR(50),
installation_date DATE,
FOREIGN KEY (connection_id) REFERENCES Connection(connection_id));
```

Inserting Values into Meter:

```
INSERT INTO Meter (connection_id, meter_number, installation_date)
VALUES
(1,'MTR1001','2023-02-01'),
(2,'MTR1002','2023-02-02'),
(3,'MTR1003','2023-02-03'),
(4,'MTR1004','2023-02-04'),
.
.
```

SELECT * FROM Meter;

OUTPUT:

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

meter_id	connection_id	meter_number	installation_date
1	1	MTR1001	2023-02-01
2	2	MTR1002	2023-02-02
3	3	MTR1003	2023-02-03
4	4	MTR1004	2023-02-04
5	5	MTR1005	2023-02-05
6	6	MTR1006	2023-02-06
7	7	MTR1007	2023-02-07
8	8	MTR1008	2023-02-08
9	9	MTR1009	2023-02-09
10	10	MTR1010	2023-02-10
11	11	MTR1011	2023-02-11
12	12	MTR1012	2023-02-12
13	13	MTR1013	2023-02-13
14	14	MTR1014	2023-02-14
15	15	MTR1015	2023-02-15
NULL	NULL	NULL	NULL

4) Create Table Meter Readings

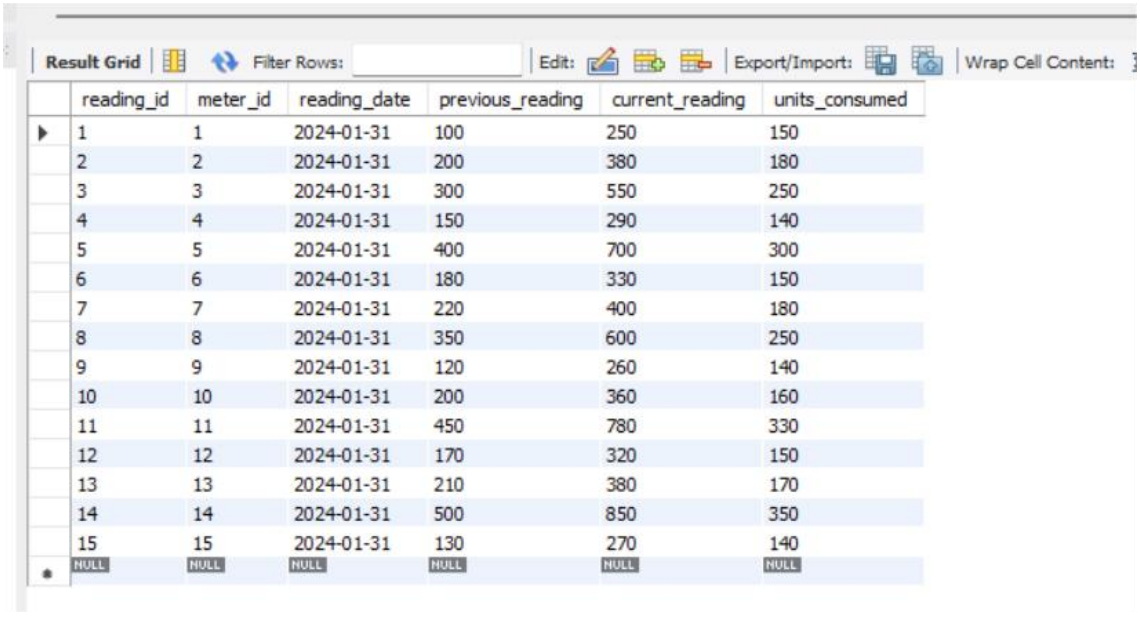
```
CREATE TABLE Meter_Readings (
reading_id INT PRIMARY KEY AUTO_INCREMENT,
meter_id INT,
reading_date DATE,
previous_reading INT,
current_reading INT,
units_consumed INT,
FOREIGN KEY (meter_id) REFERENCES Meter(meter_id)
);
```

Inserting Values into Meter_ Readings:

```
INSERT INTO Meter_Readings
(meter_id, reading_date, previous_reading, current_reading, units_consumed) VALUES
(1,'2024-01-31',100,250,150),
(2,'2024-01-31',200,380,180),
(3,'2024-01-31',300,550,250),
(4,'2024-01-31',150,290,140),
.
.
```

SELECT * FROM Meter_Readings;

OUTPUT:



Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

reading_id	meter_id	reading_date	previous_reading	current_reading	units_consumed
1	1	2024-01-31	100	250	150
2	2	2024-01-31	200	380	180
3	3	2024-01-31	300	550	250
4	4	2024-01-31	150	290	140
5	5	2024-01-31	400	700	300
6	6	2024-01-31	180	330	150
7	7	2024-01-31	220	400	180
8	8	2024-01-31	350	600	250
9	9	2024-01-31	120	260	140
10	10	2024-01-31	200	360	160
11	11	2024-01-31	450	780	330
12	12	2024-01-31	170	320	150
13	13	2024-01-31	210	380	170
14	14	2024-01-31	500	850	350
15	15	2024-01-31	130	270	140
NULL	NULL	NULL	NULL	NULL	NULL

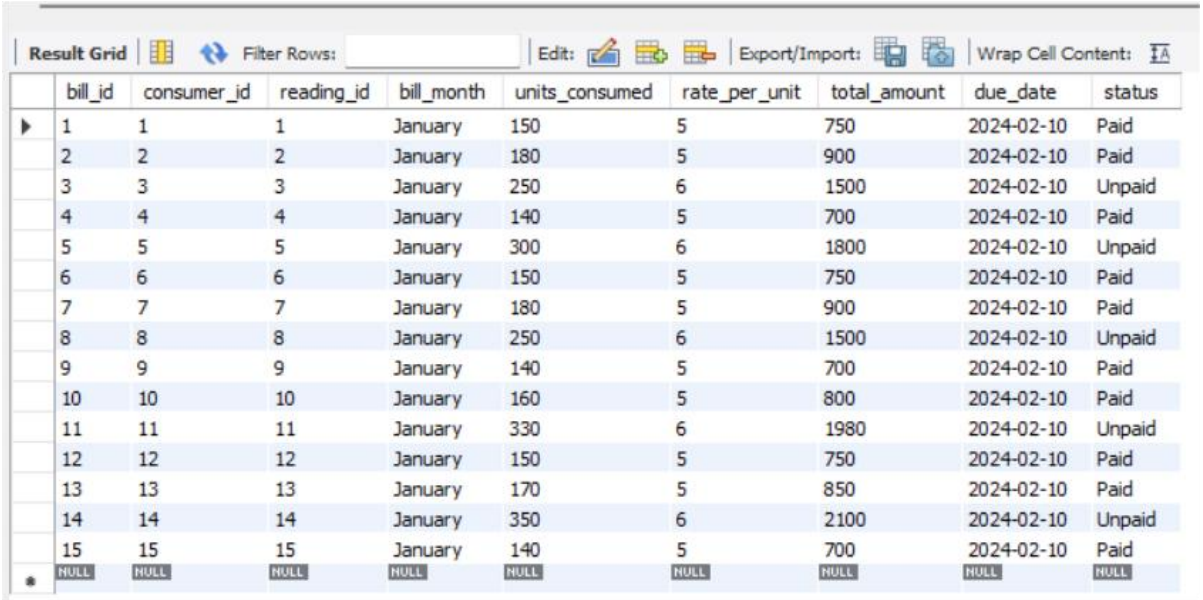
5) Create Table Bills

```
CREATE TABLE Bills (
bill_id INT PRIMARY KEY AUTO_INCREMENT,
consumer_id INT,
 reading_id INT,
 bill_month VARCHAR(20),
 units_consumed INT,
 rate_per_unit FLOAT,
 total_amount FLOAT,
 due_date DATE,
 status VARCHAR(20),
 FOREIGN KEY (consumer_id) REFERENCES Consumers(consumer_id),
 FOREIGN KEY (reading_id) REFERENCES Meter_Readings(reading_id)
 );
```

Inserting Values into Bills:

```
INSERT INTO Bills
(consumer_id, reading_id, bill_month, units_consumed, rate_per_unit, total_amount, due_date,
status) VALUES
(1,1,'January',150,5,750,'2024-02-10','Paid'),
(2,2,'January',180,5,900,'2024-02-10','Paid'),
(3,3,'January',250,6,1500,'2024-02-10','Unpaid'),
(4,4,'January',140,5,700,'2024-02-10','Paid'),
.
.
SELECT * FROM Bills;
```

OUTPUT:



Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

bill_id	consumer_id	reading_id	bill_month	units_consumed	rate_per_unit	total_amount	due_date	status
1	1	1	January	150	5	750	2024-02-10	Paid
2	2	2	January	180	5	900	2024-02-10	Paid
3	3	3	January	250	6	1500	2024-02-10	Unpaid
4	4	4	January	140	5	700	2024-02-10	Paid
5	5	5	January	300	6	1800	2024-02-10	Unpaid
6	6	6	January	150	5	750	2024-02-10	Paid
7	7	7	January	180	5	900	2024-02-10	Paid
8	8	8	January	250	6	1500	2024-02-10	Unpaid
9	9	9	January	140	5	700	2024-02-10	Paid
10	10	10	January	160	5	800	2024-02-10	Paid
11	11	11	January	330	6	1980	2024-02-10	Unpaid
12	12	12	January	150	5	750	2024-02-10	Paid
13	13	13	January	170	5	850	2024-02-10	Paid
14	14	14	January	350	6	2100	2024-02-10	Unpaid
15	15	15	January	140	5	700	2024-02-10	Paid
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

6) **Create Table Payments**

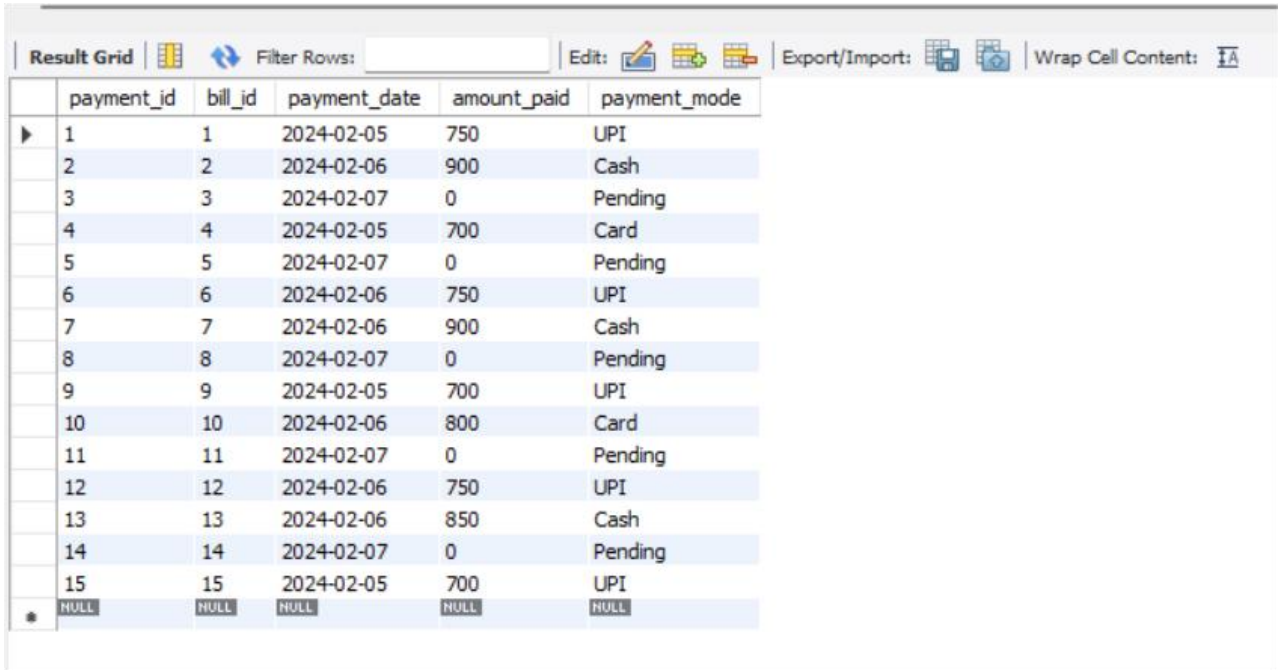
```
CREATE TABLE Payments (
   payment_id INT PRIMARY KEY AUTO_INCREMENT,
   bill_id INT,
   payment_date DATE,
   amount_paid FLOAT,
   payment_mode VARCHAR(50),
   FOREIGN KEY (bill_id) REFERENCES Bills(bill_id)
);
```

Inserting Values Payments:

```
INSERT INTO Payments (bill_id, payment_date, amount_paid, payment_mode) VALUES
(1,'2024-02-05',750,'UPI'),
(2,'2024-02-06',900,'Cash'),
(3,'2024-02-07',0,'Pending'),
(4,'2024-02-05',700,'Card'),
.
.
```

SELECT * FROM Payments;

OUTPUT:



Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

payment_id	bill_id	payment_date	amount_paid	payment_mode
1	1	2024-02-05	750	UPI
2	2	2024-02-06	900	Cash
3	3	2024-02-07	0	Pending
4	4	2024-02-05	700	Card
5	5	2024-02-07	0	Pending
6	6	2024-02-06	750	UPI
7	7	2024-02-06	900	Cash
8	8	2024-02-07	0	Pending
9	9	2024-02-05	700	UPI
10	10	2024-02-06	800	Card
11	11	2024-02-07	0	Pending
12	12	2024-02-06	750	UPI
13	13	2024-02-06	850	Cash
14	14	2024-02-07	0	Pending
15	15	2024-02-05	700	UPI
NULL	NULL	NULL	NULL	NULL

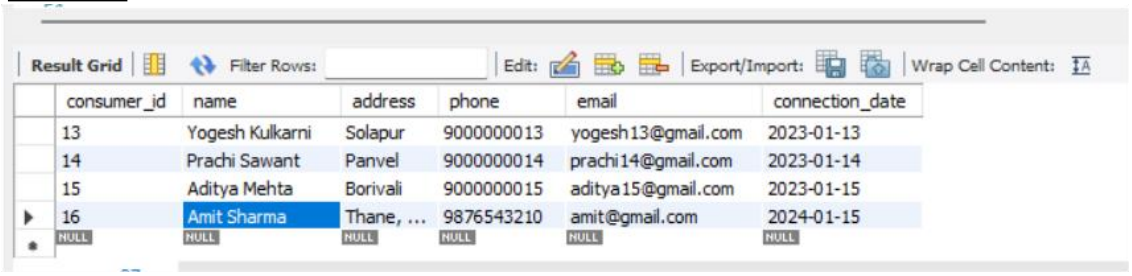
BASIC QUESTIONS

1. **Create a New Consumer Record – (‘Amit Sharma', 'Thane,maharashra', '9876543210', ‘amit@gmail.com’, '2024-01-15’)**

```
INSERT INTO consumers
(name, address, phone, email, connection_date)
    VALUES
 ('Amit Sharma', 'Thane, Maharashtra', '9876543210', 'amit@gmail.com', '2024-01-15');
```

SELECT * FROM Books;

OUTPUT:



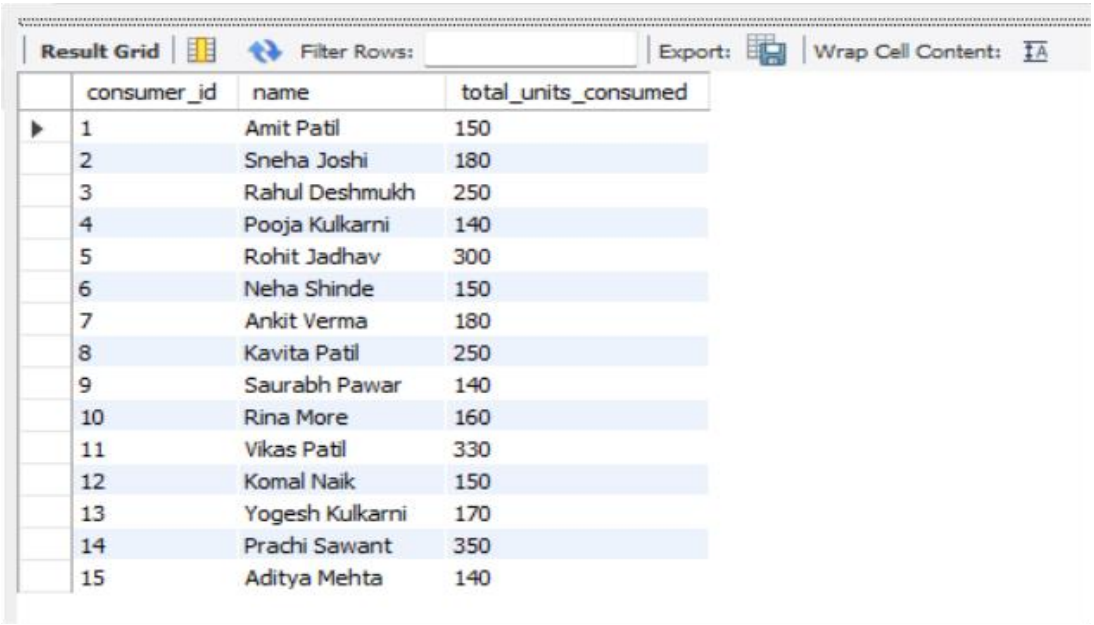
Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

consumer_id	name	address	phone	email	connection_date
13	Yogesh Kulkarni	Solapur	9000000013	yogesh13@gmail.com	2023-01-13
14	Prachi Sawant	Panvel	9000000014	prachi14@gmail.com	2023-01-14
15	Aditya Mehta	Borivali	9000000015	aditya15@gmail.com	2023-01-15
16	Amit Sharma	Thane, ...	9876543210	amit@gmail.com	2024-01-15
NULL	NULL	NULL	NULL	NULL	NULL

2.List UNITS CONSUMED BY CUSTOMERS

```
SELECT c.consumer_id, c.name,                    SUM(b.units_consumed)
AS total_units_consumed                          FROM Consumers c
JOIN Bills b ON c.consumer_id = b.consumer_id
GROUP BY c.consumer_id, c.name;
```

OUTPUT:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

consumer_id	name	total_units_consumed
1	Amit Patil	150
2	Sneha Joshi	180
3	Rahul Deshmukh	250
4	Pooja Kulkarni	140
5	Rohit Jadhav	300
6	Neha Shinde	150
7	Ankit Verma	180
8	Kavita Patil	250
9	Saurabh Pawar	140
10	Rina More	160
11	Vikas Patil	330
12	Komal Naik	150
13	Yogesh Kulkarni	170
14	Prachi Sawant	350
15	Aditya Mehta	140

3.List Consumers Who have Consumed More than 300 units

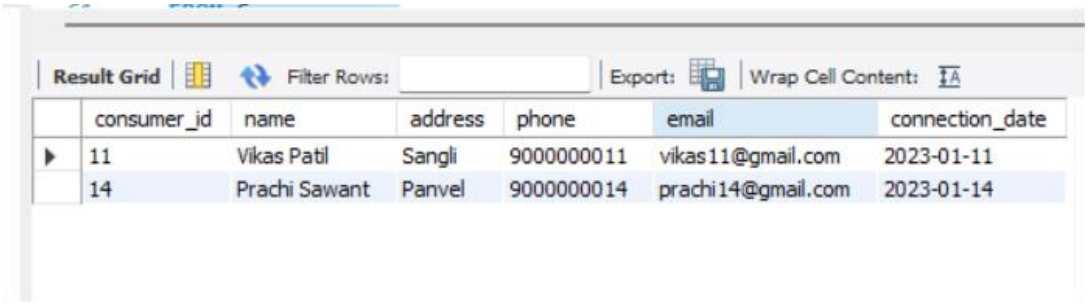
```
SELECT DISTINCT c.*

FROM Consumers c

JOIN Bills b ON c.consumer_id = b.consumer_id

WHERE b.units_consumed > 300;
```

OUTPUT:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

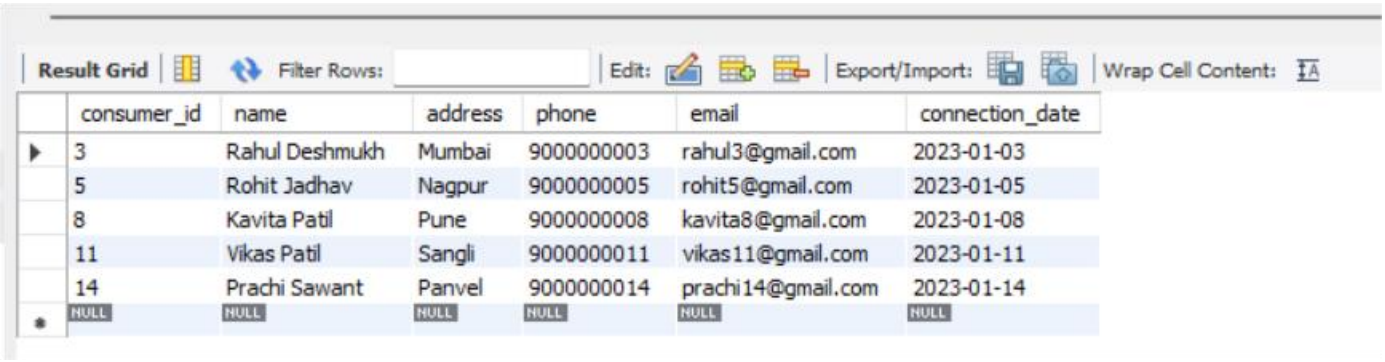
consumer_id	name	address	phone	email	connection_date
11	Vikas Patil	Sangli	9000000011	vikas11@gmail.com	2023-01-11
14	Prachi Sawant	Panvel	9000000014	prachi14@gmail.com	2023-01-14

SUB-QUERIES

1. **Consumers who consumed more than average units**

```
SELECT *
FROM Consumer
WHERE consumer_id IN (SELECT consumer_id FROM Bill WHERE units_consumed >
(SELECT AVG(units_consumed) FROM Bill));
```

OUTPUT:

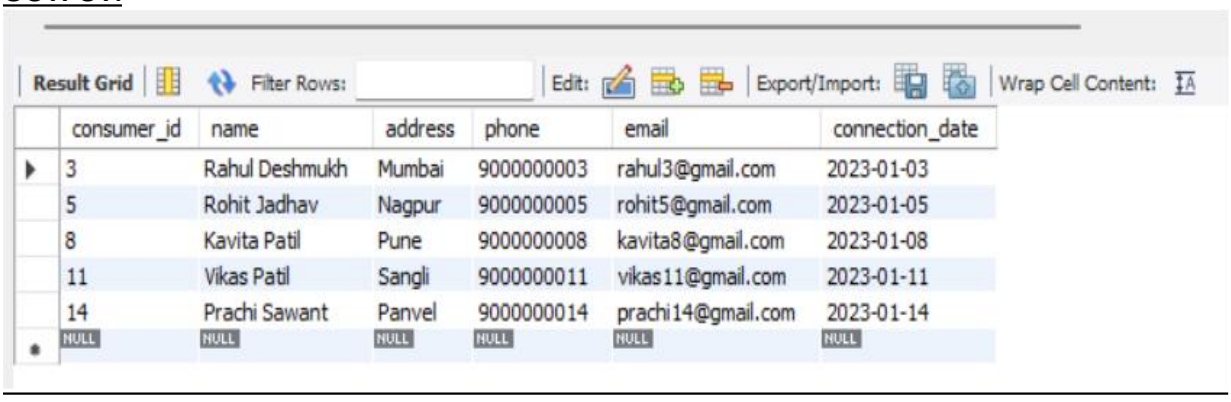


consumer_id	name	address	phone	email	connection_date
3	Rahul Deshmukh	Mumbai	9000000003	rahul3@gmail.com	2023-01-03
5	Rohit Jadhav	Nagpur	9000000005	rohit5@gmail.com	2023-01-05
8	Kavita Patil	Pune	9000000008	kavita8@gmail.com	2023-01-08
11	Vikas Patil	Sangli	9000000011	vikas11@gmail.com	2023-01-11
14	Prachi Sawant	Panvel	9000000014	prachi14@gmail.com	2023-01-14
NULL	NULL	NULL	NULL	NULL	NULL

2. **Write a query to find the Consumers Whose PAYMENT STATUS is “paid”**

```
SELECT *FROM Consumers
WHERE consumer_id IN (SELECT consumer_id  FROM Bill WHERE bill_id NOT IN
(                                                                SELECT bill_id FROM
Payment WHERE status = 'Paid' ));
```

OUTPUT:

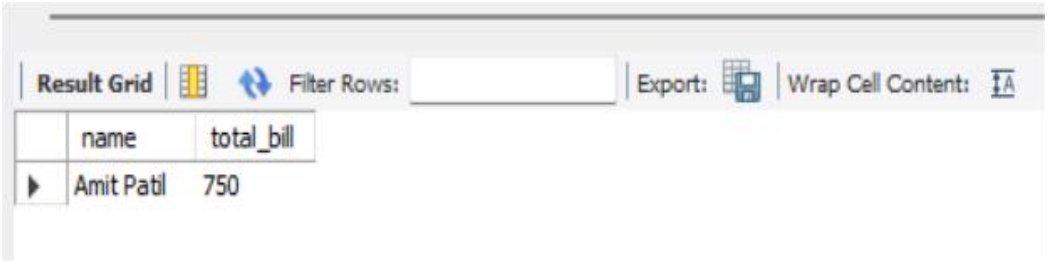


consumer_id	name	address	phone	email	connection_date
3	Rahul Deshmukh	Mumbai	9000000003	rahul3@gmail.com	2023-01-03
5	Rohit Jadhav	Nagpur	9000000005	rohit5@gmail.com	2023-01-05
8	Kavita Patil	Pune	9000000008	kavita8@gmail.com	2023-01-08
11	Vikas Patil	Sangli	9000000011	vikas11@gmail.com	2023-01-11
14	Prachi Sawant	Panvel	9000000014	prachi14@gmail.com	2023-01-14
NULL	NULL	NULL	NULL	NULL	NULL

3.Total Bill Amount of One Customer

SELECT name,(SELECT SUM(amount)FROMBillWHERE Bill.consumer_id = Consumer.consumer_id) AS total_billFROM ConsumerWHERE consumer_id = 1;

OUTPUT:

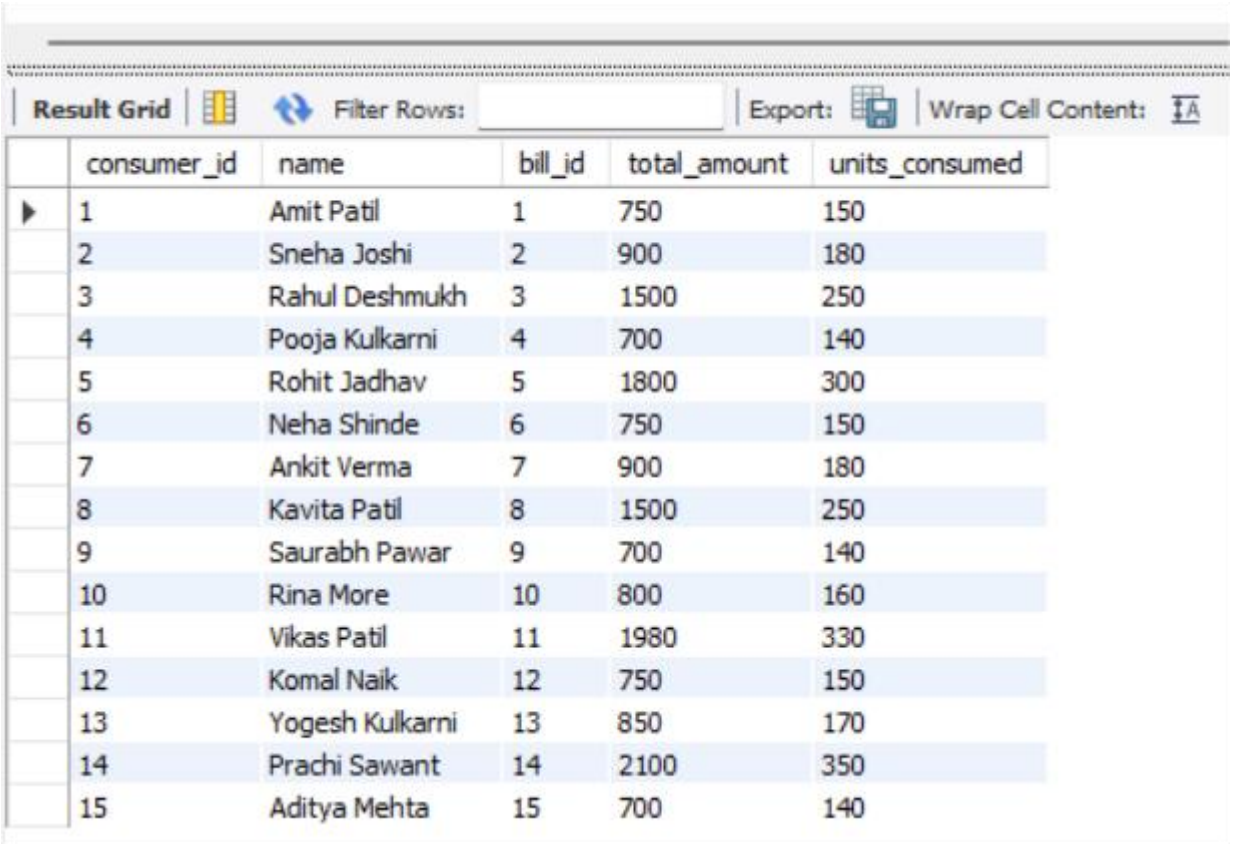


JOINS

1. Consumer details with their bill information

```
SELECT c.consumer_id, c.name, b.bill_id, b.amount, b.units_consumed
FROM Consumer c
INNER JOIN Bill b
ON c.consumer_id = b.consumer_id;
```

OUTPUT:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

consumer_id	name	bill_id	total_amount	units_consumed
1	Amit Patil	1	750	150
2	Sneha Joshi	2	900	180
3	Rahul Deshmukh	3	1500	250
4	Pooja Kulkarni	4	700	140
5	Rohit Jadhav	5	1800	300
6	Neha Shinde	6	750	150
7	Ankit Verma	7	900	180
8	Kavita Patil	8	1500	250
9	Saurabh Pawar	9	700	140
10	Rina More	10	800	160
11	Vikas Patil	11	1980	330
12	Komal Naik	12	750	150
13	Yogesh Kulkarni	13	850	170
14	Prachi Sawant	14	2100	350
15	Aditya Mehta	15	700	140

2. All consumers + their bills

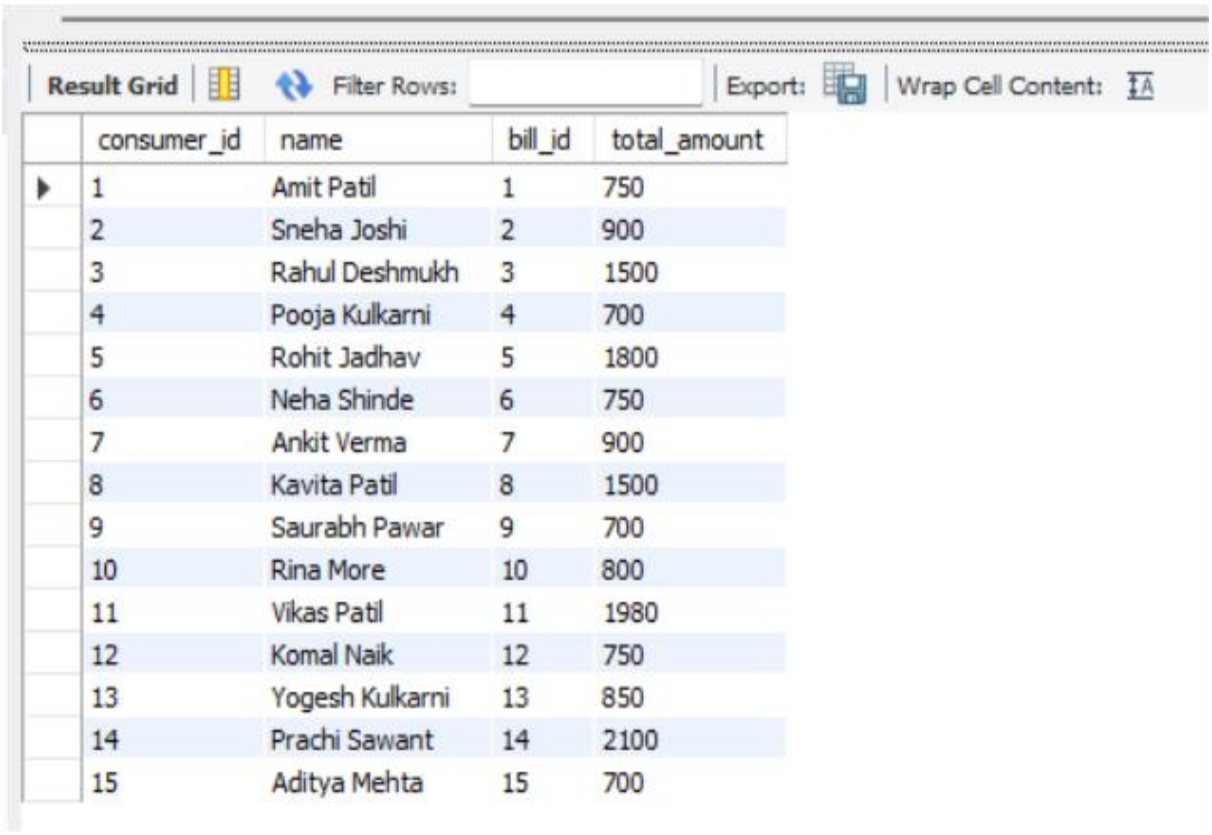
```
SELECT c.consumer_id, c.name, b.bill_id, b.amount

FROM Consumer c

LEFT JOIN Bill b

ON c.consumer_id = b.consumer_id;
```

OUTPUT:



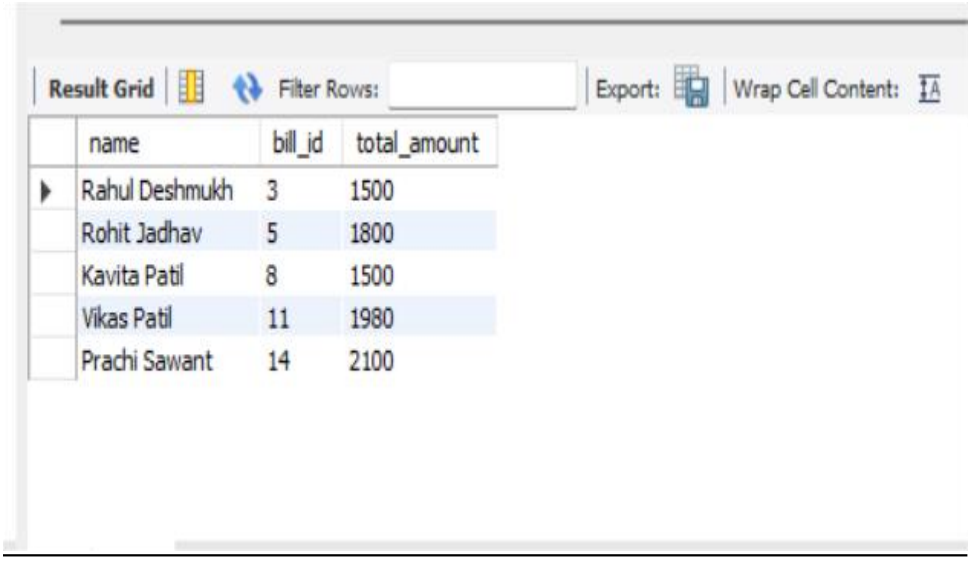
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

consumer_id	name	bill_id	total_amount
1	Amit Patil	1	750
2	Sneha Joshi	2	900
3	Rahul Deshmukh	3	1500
4	Pooja Kulkarni	4	700
5	Rohit Jadhav	5	1800
6	Neha Shinde	6	750
7	Ankit Verma	7	900
8	Kavita Patil	8	1500
9	Saurabh Pawar	9	700
10	Rina More	10	800
11	Vikas Patil	11	1980
12	Komal Naik	12	750
13	Yogesh Kulkarni	13	850
14	Prachi Sawant	14	2100
15	Aditya Mehta	15	700

3. Bills with unpaid status

```
SELECT c.name, b.bill_id, b.amount
FROM Consumer c
LEFT JOIN Bill b ON c.consumer_id = b.consumer_id
LEFT JOIN Payment p ON b.bill_id = p.bill_id
WHERE p.status IS NULL OR p.status = 'Unpaid';
```

OUTPUT:



name	bill_id	total_amount
Rahul Deshmukh	3	1500
Rohit Jadhav	5	1800
Kavita Patil	8	1500
Vikas Patil	11	1980
Prachi Sawant	14	2100

CONCLUSION

The Electricity Billing System project demonstrates the effective use of MySQL for managing electricity consumers, connections, meter details, and meter readings. The system automates billing-related operations, ensuring accurate calculation of electricity usage and improved data organization.

This project applies key database concepts such as table relationships, CRUD operations, joins, and subqueries to handle real-world billing scenarios efficiently. Overall, it reduces manual work, improves accuracy, and provides a solid understanding of relational database design.